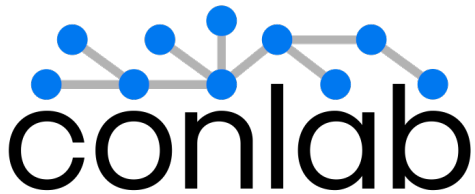
Policy Iteration for Linear Quadratic Games with Stochastic Parameters

Benjamin Gravell, Karthik Ganapathy, and Tyler Summers



59th IEEE Conference on Decision and Control
Jeju Island, Republic of Korea - December 14-18 2020

Motivation

State regulation - a fundamental problem in control

Linear time-invariant (LTI) models admit tractable and theoretically rigorous optimal controllers

In practice, no LTI model is perfect

- Unstructured uncertainty
  - Nonlinearities
  - State-space reductions
- Structured uncertainty
  - Parameter misspecification

How can we design controllers **robust** to these **uncertainties**?

Robust stabilization

Two sets of tools to extend and robustify LTI models

- Unstructured uncertainty
  - $\mathcal{H}_\infty$ optimal control via dynamic game formulation
  - $\sim$ "Adversarial training" in machine learning
- Structured uncertainty
  - Stochastic parameters / multiplicative noise
  - $\sim$ "Temporal domain randomization" in machine learning

We propose their **combination** in a single framework

Stochastic parameter systems

Linear stochastic parameter system with adversary:

$$x_{t+1} = \widetilde{A}_t x_t + \widetilde{B}_t u_t + \widetilde{C}_t v_t$$

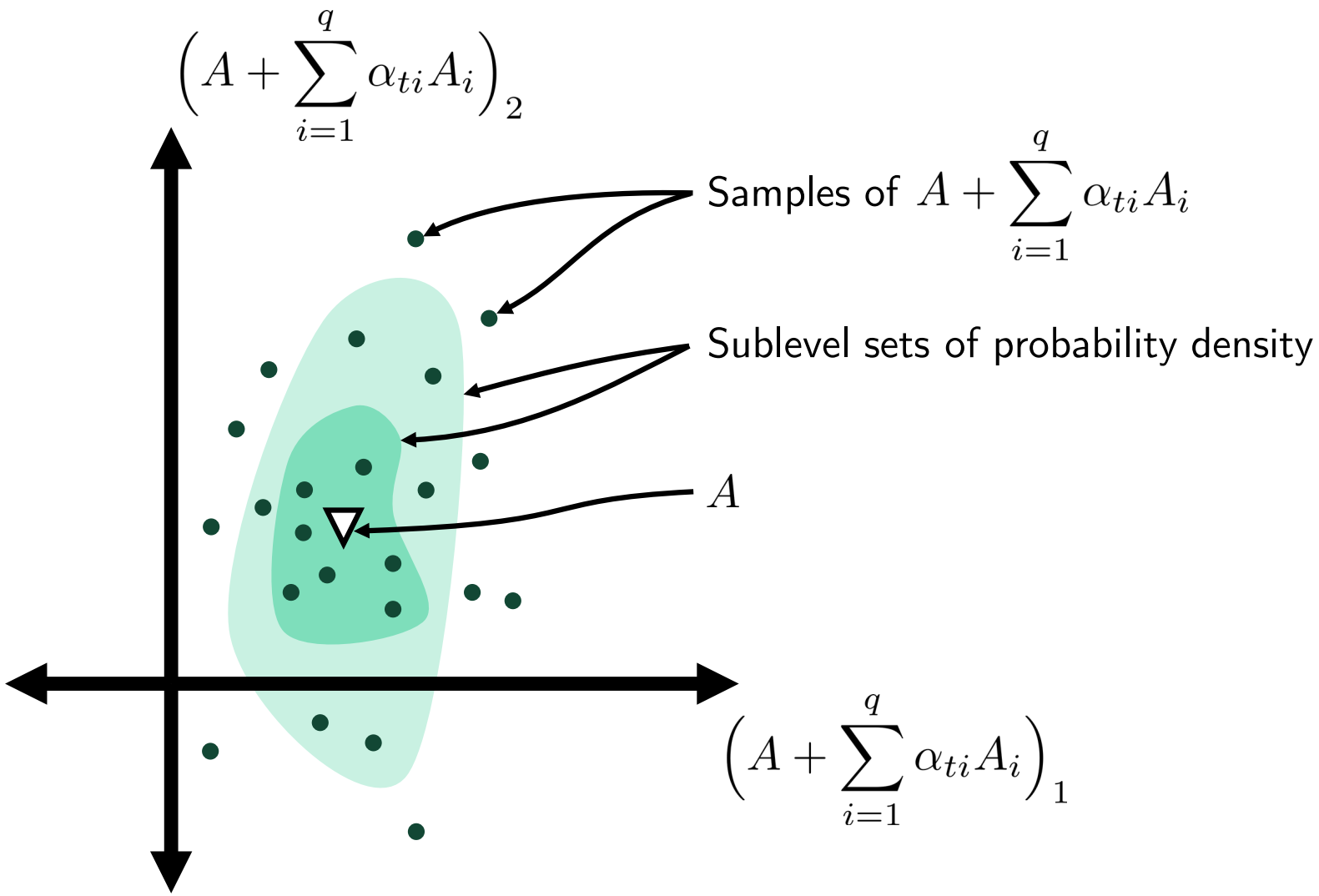
- x state
- u control input
- v adversary input

Random matrices $\widetilde{A}_t, \widetilde{B}_t, \widetilde{C}_t$ are i.i.d. across time, decompose as

$$\widetilde{A}_t = A + \sum_{i=1}^{q} \alpha_{ti} A_i, \quad \widetilde{B}_t = B + \sum_{j=1}^{r} \beta_{tj} B_j, \quad \widetilde{C}_t = C + \sum_{k=1}^{s} \gamma_{tk} C_k,$$

- A, B, C encode mean dynamics
- α_{ti}, β_{tj}, γ_{tk} are zero-mean, mutually independent
- A_i, B_j, C_k encode parameter uncertainty patterns
- Equivalently: state- and input-multiplicative noise

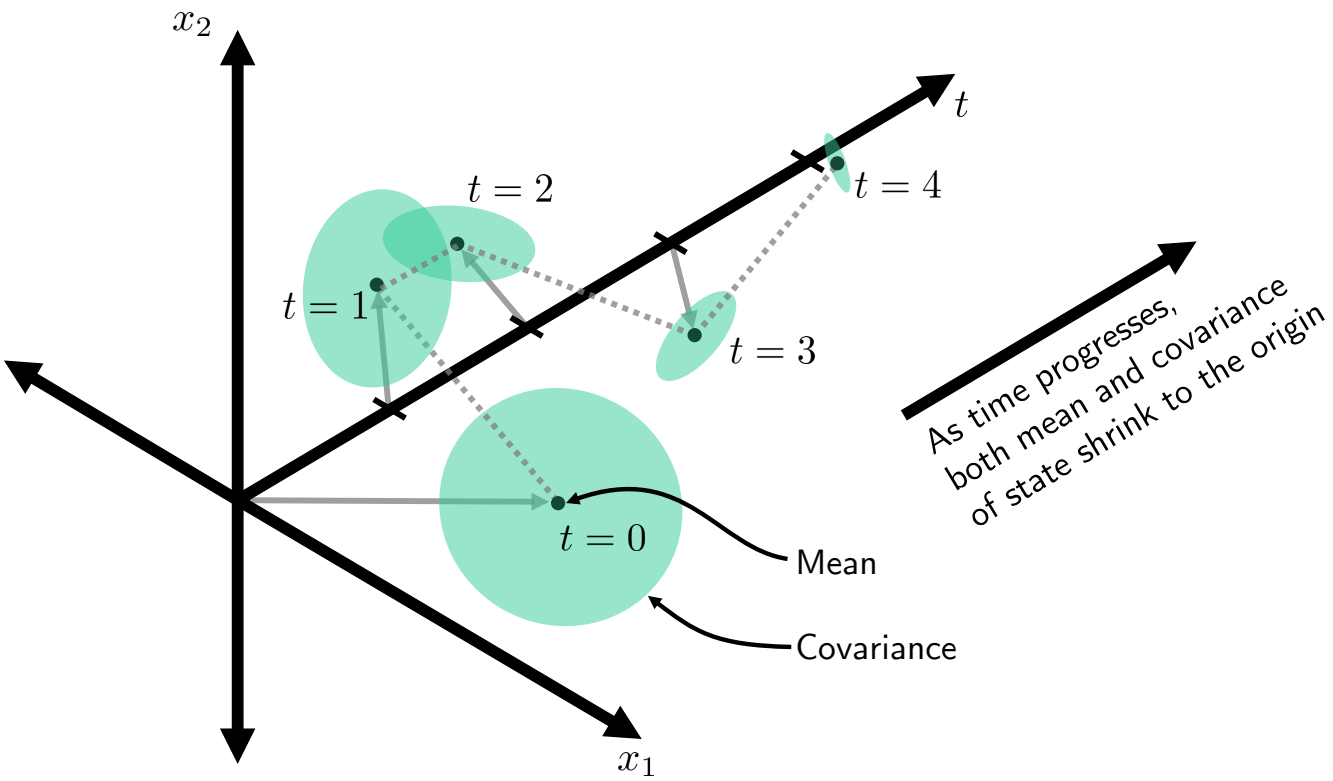
Stochastic parameter systems



Mean-square stability [1, 2]

A stochastic system is **mean-square stable** if and only if

$$\lim_{t \to \infty} \mathbb{E}\left[x_t x_t^\intercal\right] = 0 \; \forall \; \|x_0\| < \infty$$



Linear quadratic game with stochastic parameters

Linear quadratic zero-sum game with stochastic parameters (SLQ game)

$$\begin{aligned} \underset{\pi_u \in \Pi_u}{\text{minimize}} \ \underset{\pi_v \in \Pi_v}{\text{maximize}} \quad & \mathop{\mathbb{E}}_{\widetilde{A},\widetilde{B},\widetilde{C}} \sum_{t=0}^{\infty} c(x_t, u_t, v_t), \\ \text{subject to} \quad & x_{t+1} = \widetilde{A}_t x_t + \widetilde{B}_t u_t + \widetilde{C}_t v_t, \end{aligned}$$

- Quadratic stage costs $c(x_t, u_t, v_t) = x_t^\intercal Q x_t + u_t^\intercal R u_t - v_t^\intercal S v_t$
  - $Q \succ 0 \Rightarrow$ convex in x
  - $R \succ 0 \Rightarrow$ convex in u
  - $S \succ 0 \Rightarrow$ concave in v
- Linear dynamics with stochastic parameters

Solution of SLQ game

Solve the generalized game algebraic Riccati equation (GARE)

$$P^* = H^*_{xx} - \begin{bmatrix} H^*_{ux} \\ H^*_{vx} \end{bmatrix}^\intercal \begin{bmatrix} H^*_{uu} & H^*_{uv} \\ H^*_{vu} & H^*_{vv} \end{bmatrix}^{-1} \begin{bmatrix} H^*_{ux} \\ H^*_{vx} \end{bmatrix},$$

where

$$H^* = \begin{bmatrix} Q & 0 & 0 \\ 0 & R & 0 \\ 0 & 0 & -S \end{bmatrix} + \mathop{\mathbb{E}}_{\widetilde{A},\widetilde{B},\widetilde{C}} \left(\begin{bmatrix} \widetilde{A} & \widetilde{B} & \widetilde{C} \end{bmatrix}^\intercal P^* \begin{bmatrix} \widetilde{A} & \widetilde{B} & \widetilde{C} \end{bmatrix} \right)$$

Linear state feedback $u_t = K^* x_t$ and $v_t = L^* x_t$ where

$$K^* = (H^*_{uu} - H^*_{uv} {H^*_{vv}}^{-1} H^*_{vu})^{-1} (H^*_{uv} {H^*_{vv}}^{-1} H^*_{vx} - H^*_{ux}),$$
$$L^* = (H^*_{vv} - H^*_{vu} {H^*_{uu}}^{-1} H^*_{uv})^{-1} (H^*_{vu} {H^*_{uu}}^{-1} H^*_{ux} - H^*_{vx}).$$

Solving the Riccati equation

Existing approaches require knowledge of the problem data

- Value iteration
- Semidefinite programming (SDP)

Alternative: **policy iteration**

- Dynamic programming technique
- Alternates between policy evaluation and policy improvement
- Gains converge **quadratically** in any matrix norm to the optimum [3]
- Extends readily to the model-free setting

Exact policy iteration for SLQ games

Input: Ms-stabilizing policy gains (K^0, L^0), convergence threshold $\epsilon > 0$.

1: Initialize $P^0 = 0$, $P^1 = \infty$, $k = 0$

2: **while** $\|P^{k+1} - P^k\| > \epsilon$ **do**

3: **Policy Evaluation:**

Compute value of current policies by solving Lyapunov equation

$$P_{KL}^k = Q_{KL}^k + \mathcal{F}_{KL}^k(P_{KL}^k)$$

4: **Policy Improvement:**

Update policies according to

$$K^{k+1} = (H_{uu} - H_{uv}H_{vv}^{-1}H_{vu})^{-1}(H_{uv}H_{vv}^{-1}H_{vx} - H_{ux})$$
$$L^{k+1} = (H_{vv} - H_{vu}H_{uu}^{-1}H_{uv})^{-1}(H_{vu}H_{uu}^{-1}H_{ux} - H_{vx})$$

with $H = H_{KL}^k$ defined in terms of P_{KL}^k.

5: $k \leftarrow k + 1$

Output: Approximately optimal policy pair (K^{k+1}, L^{k+1})

Approximate policy iteration

How can we apply policy iteration when we do not know the problem data A, B, C, etc.?

Key insight: the policy updates can be executed with H alone

Key idea: estimate H directly from observed state-input trajectories

State-action value estimation

State-value function

$$V_{KL}(x) = \mathop{\mathbb{E}}_{\alpha,\beta,\gamma} \sum_{t=0}^{\infty} x_t^\intercal (Q + K^\intercal R K - L^\intercal S L) x_t, \quad x = x_0$$

State-action value function

$$\mathcal{Q}_{KL}(x,u,v) = c(x,u,v) + \mathop{\mathbb{E}}_{\widetilde{A},\widetilde{B},\widetilde{C}} V_{KL}(\widetilde{A}x + \widetilde{B}u + \widetilde{C}v)$$

$$= \begin{bmatrix} x \\ u \\ v \end{bmatrix}^\intercal \begin{bmatrix} H_{xx} & H_{xu} & H_{xv} \\ H_{ux} & H_{uu} & H_{uv} \\ H_{vx} & H_{vu} & H_{vv} \end{bmatrix} \begin{bmatrix} x \\ u \\ v \end{bmatrix}$$

Characterized by matrix H

State-action value estimation

Use symmetric vectorization to convert $\mathcal{Q}_{KL}$ to a linear architecture:

$$\mathcal{Q}_{KL}(z) = \phi(z)^\intercal \Theta$$

where

$$\begin{aligned} z &= \begin{bmatrix} x^\intercal & u^\intercal & v^\intercal \end{bmatrix}^\intercal && \text{(concatenated state-input vector)} \\ \Theta &= \operatorname{svec}(H_{KL}) && \text{(parameter vector)} \\ \phi(z) &= \operatorname{svec}\left(zz^\intercal\right) && \text{(feature map)} \end{aligned}$$

Goal: estimate Θ from data

We have the cost relationship

$$c(z_t) = \phi(z_t)^\intercal \Theta - \mathbb{E}\left[\phi(w_{t+1})^\intercal \Theta\right]$$

where $w = \begin{bmatrix} x^\intercal & (Kx)^\intercal & (Lx)^\intercal \end{bmatrix}^\intercal$

Cannot find $\mathbb{E}\left[\phi(w_{t+1})^\intercal \Theta\right]$ exactly $\Rightarrow$ replace with transition samples:

$$c(z_t) = \left(\phi(z_t) - \phi(w_{t+1})\right)^\intercal \Theta + e_t,$$

where the error is

$$e_t = \phi(w_{t+1})^\intercal \Theta - \mathbb{E}\left[\phi(w_{t+1})^\intercal \Theta\right].$$

State-action value estimation

Stacking ℓ observations yields

$$Y = Z\Theta + E$$

where

$$Y = \begin{bmatrix} c(z_0) \\ c(z_1) \\ \vdots \\ c(z_\ell) \end{bmatrix}, \quad Z = \begin{bmatrix} (\phi(z_0) - \phi(w_1))^\intercal \\ (\phi(z_1) - \phi(w_2))^\intercal \\ \vdots \\ (\phi(z_\ell) - \phi(w_{\ell+1}))^\intercal \end{bmatrix}, \quad E = \begin{bmatrix} e_0 \\ e_1 \\ \vdots \\ e_\ell \end{bmatrix}.$$

Applying least-squares gives the **LSADP estimator**

$$\widehat{\Theta} = (Z^\intercal Z)^{-1} Z^\intercal Y$$

State-action value estimation

Issue: LSADP estimator is **biased**

- Errors e_t are correlated with the observations $c(z_t)$

Instead, use the **LSTDQ estimator** [4]:

$$\widehat{\Theta} = \left(\sum_{t=1}^{\ell} \phi(z_t)(\phi(z_t) - \phi(w_{t+1})^{\intercal} \right)^{\dagger} \sum_{t=1}^{\ell} \phi(z_t) c_t.$$

Now put together this estimator with the policy updates to obtain approximate policy iteration

Approximate policy iteration for SLQ games

Input: Gains (K^0, L^0), number of iterations N, rollout length ℓ, variances σ_u^2, σ_v^2.

1: Initialize $k = 0$

2: **while** $k < N$ **do**

3: **Approximate Policy Evaluation:**

4: Generate inputs $u_t = Kx_t + u_{e,t}$, $v_t = Lx_t + v_{e,t}$ with exploration noises $u_{e,t} \sim \mathcal{N}(0, \sigma_u^2 I)$, $v_{e,t} \sim \mathcal{N}(0, \sigma_v^2 I)$.

5: Collect rollout $\{z_t, c_t\}_{t=0}^{\ell+1}$ by exciting the system with inputs u_t, v_t.

6: Compute augmented rollout $\{z_t, w_t, c_t\}_{t=0}^{\ell+1}$ under policy pair (K^k, L^k).

7: Compute $\widehat{H} = \widehat{H}_{KL}^k$ using LSTDQ estimation as

$$\widehat{\Theta}_{KL}^k = \left(\sum_{t=1}^{\ell} \phi(z_t)(\phi(z_t) - \phi(w_{t+1})^\intercal \right)^\dagger \sum_{t=1}^{\ell} \phi(z_t) c_t$$

$$\widehat{H}_{KL}^k = \operatorname{smat}(\widehat{\Theta})$$

8: **Policy Improvement:** Update the policy pair according to

$$K^{k+1} = (\widehat{H}_{uu} - \widehat{H}_{uv}\widehat{H}_{vv}^{-1}\widehat{H}_{vu})^{-1}(\widehat{H}_{uv}\widehat{H}_{vv}^{-1}\widehat{H}_{vx} - \widehat{H}_{ux})$$

$$L^{k+1} = (\widehat{H}_{vv} - \widehat{H}_{vu}\widehat{H}_{uu}^{-1}\widehat{H}_{uv})^{-1}(\widehat{H}_{vu}\widehat{H}_{uu}^{-1}\widehat{H}_{ux} - \widehat{H}_{vx})$$

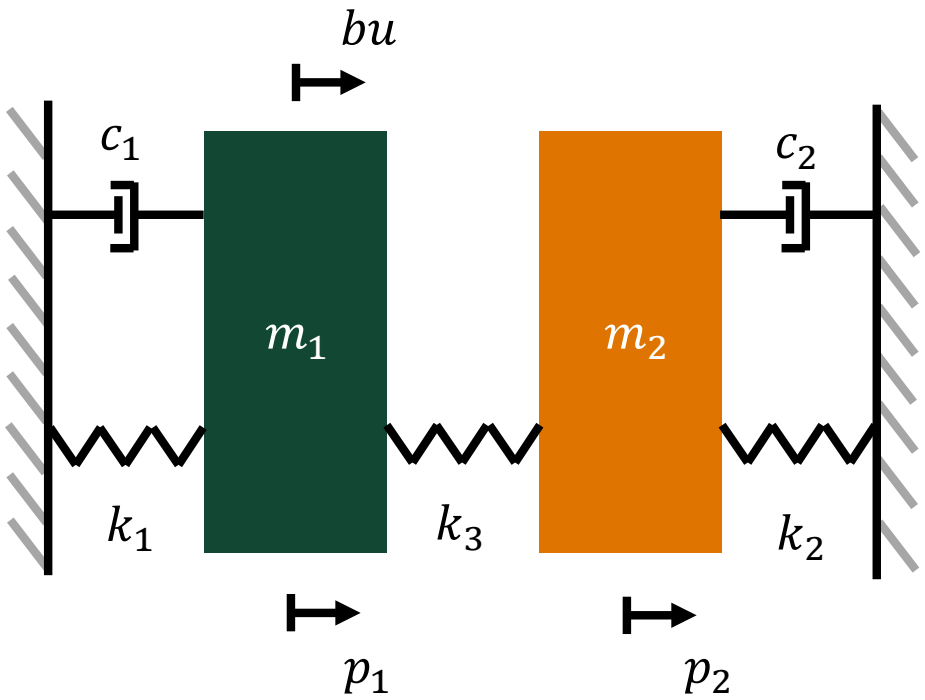
9: $k \leftarrow k + 1$

Output: Approximately optimal policy pair (K^k, L^k)

Numerical example: true system

Spring-mass-damper with continuous-time dynamics

$$\begin{bmatrix} \dot{p}_1 \\ \ddot{p}_1 \\ \dot{p}_2 \\ \ddot{p}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{k_1+k_3}{m_1} & -\frac{c_1}{m_1} & -\frac{k_1}{m_1} & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{k_2}{m_2} & 0 & -\frac{k_2+k_3}{m_2} & -\frac{c_2}{m_2} \end{bmatrix} \begin{bmatrix} p_1 \\ \dot{p}_1 \\ p_2 \\ \dot{p}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ b \\ 0 \\ 0 \end{bmatrix} u$$



Numerical example: nominal system

1) Neglects second mass i.e. treats it as a fixed wall:

$$A = \begin{bmatrix} 1 & \Delta t \\ -\left(\frac{k_1+k_3}{m_1}\right)\Delta t & 1-\left(\frac{c_1}{m_1}\right)\Delta t \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ b\Delta t \end{bmatrix}.$$

Uncertainty from unmodeled dynamics $\Rightarrow$ adversary with input matrix

$$C = \begin{bmatrix} 0 & \Delta t \end{bmatrix}^\intercal$$

2) Underestimates spring constant, overestimates actuator strength

Parametric uncertainty $\Rightarrow$ multiplicative noises with directions

$$A_1 = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad B_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

with variances $\sigma^2_{\alpha,1} = 0.8$ and $\sigma^2_{\beta,1} = 0.4$.

Numerical results

Apply exact policy iteration to the nominal system

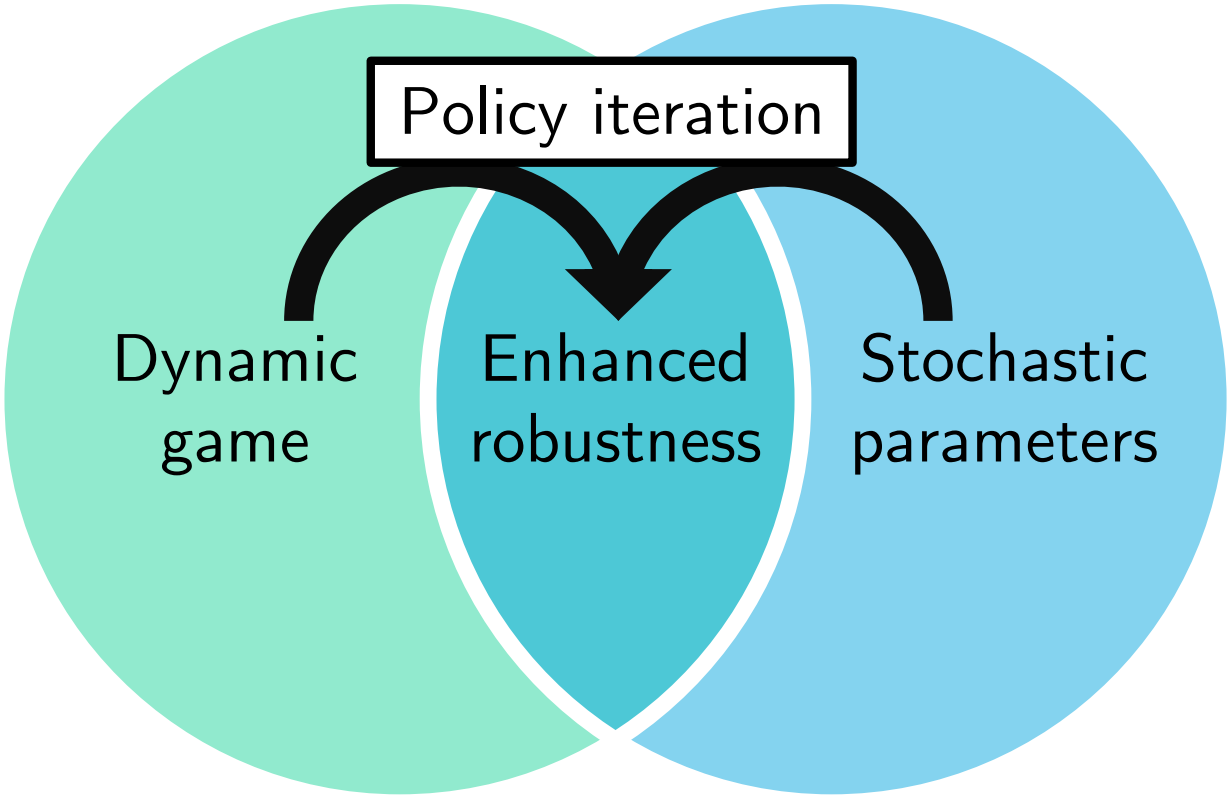
Evaluate controllers on the true system

Method	$\rho(A+BK)$	$\mathrm{Tr}(P_K)$	K
OL	1.570	∞	$\begin{bmatrix}+0.0 & +0.0 & +0.0 & +0.0\end{bmatrix}$
CE	1.031	∞	$\begin{bmatrix}-1.6 & -1.5 & +0.0 & +0.0\end{bmatrix}$
MN	1.021	∞	$\begin{bmatrix}-1.7 & -1.6 & +0.0 & +0.0\end{bmatrix}$
DG	1.018	∞	$\begin{bmatrix}-1.8 & -1.7 & +0.0 & +0.0\end{bmatrix}$
MNDG	**0.994**	**31074**	$\begin{bmatrix}-2.3 & -2.1 & +0.0 & +0.0\end{bmatrix}$
OPT	0.965	3019	$\begin{bmatrix}-2.6 & -2.0 & -2.1 & +2.9\end{bmatrix}$

Uncertainties are so severe that **only the controller which considers multiplicative noise and dynamic game (MNDG) achieves stability**

Code available on GitHub at
https://github.com/TSummersLab/policy-iteration-slq-games

Parting thoughts

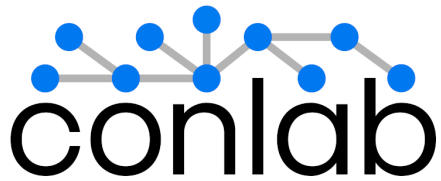


Thanks!

Contact information:

Ben Gravell	benjamin.gravell@utdallas.edu
Karthik Ganapathy	karthik.ganapathy@utdallas.edu
Tyler Summers (PI)	tyler.summers@utdallas.edu

This work also published in IEEE LCSS with CodeOcean implementation:
https://doi.org/10.1109/LCSYS.2020.3001883



This material is based on work supported by the United States Air Force Office of Scientific Research under award number FA2386-19-1-4073.

Bibliography I

[1] D. Kleinman.
On the stability of linear stochastic systems.
IEEE Transactions on Automatic Control, 14(4):429–430, August 1969.

[2] Stephen Boyd, Laurent El Ghaoui, Eric Feron, and Venkataramanan Balakrishnan.
Linear matrix inequalities in system and control theory, volume 15.
SIAM, 1994.

[3] T. Damm and D. Hinrichsen.
Newton's method for a rational matrix equation occurring in stochastic control.
Linear Algebra and its Applications, 332-334:81 – 109, 2001.

[4] Michail G. Lagoudakis and Ronald Parr.
Least-squares policy iteration.
J. Mach. Learn. Res., 4:1107–1149, December 2003.